

MP5: Kernel-Level Thread Scheduling

Phani Jyothi Kurada
UIN: 335006950
CSCE611: Operating System

Assigned Tasks

Main: FIFO Scheduler - Completed

Bonus Option 1: Correct handling of interrupts - Completed

Bonus Option 2: Round-Robin Scheduling - Completed

Bonus Option 3: Processes – Implemented & Tested (functional, non-halting behavior)

System Design

In this machine problem, we implemented scheduling and execution of multiple kernel-level threads and extended the design to support per-process address space isolation. The system integrates a basic FIFO scheduler, an optional Round-Robin scheduler using timer interrupts, and process-level address separation.

1. **User Process Context:** The kernel creates and manages user-level processes, each initialized within the `main()` function. Threads are assigned their own stacks and are linked to process-specific page directories when Option 3 (Processes) is enabled.
2. **Thread Scheduler:** This core component manages thread execution order. In the FIFO model, threads are enqueued and resumed strictly in insertion order. When Round-Robin scheduling is enabled, the scheduler uses periodic timer interrupts to preempt the currently running thread and rotate through the ready queue.
3. **FIFO Queue Management:** The scheduler maintains a ready queue that stores runnable threads in a first-in-first-out fashion. Threads voluntarily yield the CPU, allowing the next thread in the queue to execute, ensuring fairness and determinism in scheduling.
4. **Timer and Interrupt Handling (for Round-Robin):** A custom `EOQTimer` class, derived from `SimpleTimer`, was implemented to generate hardware timer interrupts (IRQ 0). Each interrupt triggers a preemption event handled by the scheduler, which performs a context switch to the next ready thread. The End-of-Interrupt (EOI) signaling ensures proper interrupt acknowledgment and re-arming.
5. **CPU Dispatch and Context Switching:** Threads are dispatched from the scheduler and executed on the CPU. In the FIFO model, threads voluntarily yield control to allow the next thread to run, whereas in the Round-Robin model, the CPU is preempted automatically via periodic timer interrupts. The `dispatch_to()` routine performs the low-level context switch by temporarily disabling interrupts, saving and restoring stack pointers, updating the `CR3` register when necessary, and transferring execution to the target thread. Once the new thread begins execution, interrupts are re-enabled to support safe preemption and further scheduling.
6. **Bonus Option 3: Processes (Per-Process Address Space Isolation):** Each process is assigned its own `PageDirectory`, ensuring memory protection and isolation. During a context switch, the kernel updates the `CR3` register with the new process's page directory physical address. This allows different processes to run in independent virtual address spaces. Process

creation, context switching, and page-table updates were verified as functional; however, the system currently exhibits non-halting behavior after all processes complete execution.

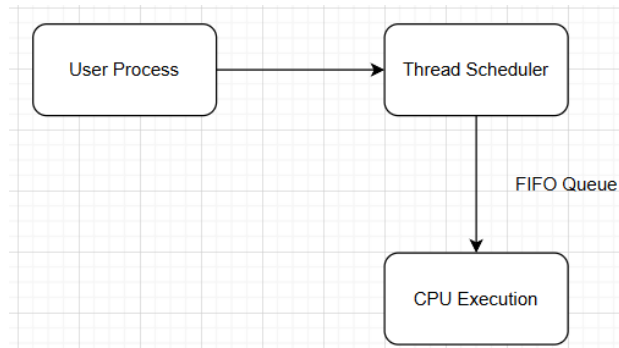


Figure 1: High Level System Design Diagram for FIFO Thread Scheduling

Code Description

Files Modified: kernel.C, thread.H, thread.C, scheduler.H, scheduler.C, threads_low.asm, machine_low.H, makefile

New Files Created: page_table.H, page_table.C, eoq_timer.H, eoq_timer.C

I added `page_table.C` and `eoq_timer.C` to the compilation pipeline in the `makefile` to ensure that the paging subsystem (for per-process address spaces) and the timer class (for Round-Robin scheduling) are properly compiled and linked into the kernel binary.

Modifications to thread.H: The Thread class was extended to support both thread termination tracking and process-specific address-space management.

- Introduced a method `char * Stack()` to safely expose the private `stack` pointer, enabling correct stack deallocation during zombie thread cleanup.
- Added a public flag `terminating` and an accessor method `is_terminating()` to mark threads pending termination and prevent them from being re-queued by the scheduler.
- Integrated a `PageDirectory *page_directory` member to associate each thread with its own page directory. This allows distinction between regular kernel threads (shared address space) and user processes (separate address spaces).
- Added helper methods `get_page_directory()` and `is_process()` to query and manage per-thread address-space state.

```
char * Stack() {
    return stack;
}

bool terminating = false;
bool is_terminating() {
    return terminating;
}
```

```
static Thread * CurrentThread();
/* Returns the currently running thread. nullptr if no thread has started yet. */

PageDirectory * get_page_directory() {
    return page_directory;
}
/* Returns the page directory for this thread/process. */

bool is_process() {
    return (page_directory != nullptr);
}
/* Returns true if this is a process (has its own address space). */

void set_esp(char * _esp) { esp = _esp; } /* Set stack pointer (for process setup) */
```

Figure 2: Modifications in thread.H showing integration of thread termination tracking and process-specific page directory management.

These modifications were essential for implementing safe deferred cleanup of terminated threads through the zombie queue mechanism and for supporting time-sliced preemption in the Round-Robin scheduler.

scheduler.C: Constructor(): Initializes two independent linked lists — one for the *ready queue* (threads waiting to execute) and another for the *zombie queue* (threads pending cleanup). The lists are represented by `ready_queue_head`, `ready_queue_tail`, `zombie_head`, and `zombie_tail`, all initialized to `nullptr`. A console message confirms the successful construction of the scheduler.

```
Scheduler::Scheduler() {
    ready_queue_head = nullptr;
    ready_queue_tail = nullptr;
    zombie_head = nullptr;
    zombie_tail = nullptr;
    Console::puts("Constructed Scheduler.\n");
}
```

Figure 3: Implementation of `Scheduler constructor()`

scheduler.C: yield(): Handles context switching by selecting the next thread from the head of the ready queue and removing it before dispatching. If the ready queue becomes empty, the system halts safely. Interrupts are disabled and re-enabled only if they were previously enabled, preventing race conditions during concurrent queue modifications. This ensures correct behavior under both cooperative and preemptive scheduling contexts. (*Bonus Option 1: Implemented proper interrupt handling during preemption.*)

```

void Scheduler::yield() {
    if (Machine::interrupts_enabled())
        Machine::disable_interrupts(); // START critical section

    if (ready_queue_head == nullptr) {
        Console::puts("No more threads to run. System halting.\n");
        while (true);
        return;
    }

    // Next thread from the ready queue
    ReadyQueueNode * next_node = ready_queue_head;
    Thread * next_thread = next_node->thread;

    // Update the head of the queue
    ready_queue_head = next_node->next;
    if (ready_queue_head == nullptr) {
        // Queue became empty
        ready_queue_tail = nullptr;
    }

    delete next_node;

    Console::puts("Yielding to thread: ");
    Console::puti(next_thread->ThreadId());
    Console::puts("\n");

    if (!Machine::interrupts_enabled())
        Machine::enable_interrupts(); // END critical section

    // Dispatch to the next thread
    Thread::dispatch_to(next_thread);
}

```

Figure 4: Implementation of yield()

scheduler.C: resume(): Adds a thread back into the ready queue so it can be scheduled for execution later. If the queue is empty, the thread becomes both the head and tail; otherwise, it is appended to the end. Before enqueueing, the method checks whether the thread is marked as `terminating()`. If so, it skips re-queueing to prevent execution of terminated threads. This method also performs cleanup of zombie threads — threads previously parked for deferred deletion — by deallocating their stacks and control blocks. Interrupts remain disabled throughout the operation to guarantee race-free updates to both ready and zombie queues.

```

void Scheduler::resume(Thread * _thread) {
    if (Machine::interrupts_enabled())
        Machine::disable_interrupts(); // START critical section

    // Do not reschedule terminated threads
    if (_thread->is_terminating()) {
        Console::puts("Not rescheduling terminating thread: ");
        Console::puti(_thread->ThreadId());
        Console::puts("\n");
        if (!Machine::interrupts_enabled())
            Machine::enable_interrupts();
        return;
    }

    Console::puts("Resuming thread: ");
    Console::puti(_thread->ThreadId());
    Console::puts("\n");

    ReadyQueueNode * new_node = new ReadyQueueNode;
    new_node->thread = _thread;
    new_node->next = nullptr;

    if (ready_queue_tail == nullptr) {
        // Queue is empty
        ready_queue_head = new_node;
        ready_queue_tail = new_node;
    } else {
        // Append to end
        ready_queue_tail->next = new_node;
        ready_queue_tail = new_node;
    }

    // Clean up zombies
    while (zombie_head != nullptr) {
        ZombieNode * dead = zombie_head;
        zombie_head = dead->next;

        Console::puts("Cleaning zombie thread: ");
        Console::puti(dead->thread->ThreadId());
        Console::puts("\n");

        delete[] (char *)dead->thread->Stack();
        delete dead->thread;
        delete dead;
    }
    zombie_tail = nullptr;

    if (!Machine::interrupts_enabled()) {
        Machine::enable_interrupts(); // END critical section
    }
}

```

Figure 5: Implementation of resume()

scheduler.C: add(): Inserts a newly created thread into the ready queue so it can be scheduled for execution. A new `ReadyQueueNode` is allocated for the thread. If the queue is currently empty, both the head and tail pointers are updated to the new node; otherwise, the node is appended to the end. Unlike `resume()`, this function is used only during thread creation and not during preemption or context switching.

```
void Scheduler::add(Thread * _thread) {
    Console::puts("Adding thread to ready queue: ");
    Console::puti(_thread->ThreadId());
    Console::puts("\n");

    ReadyQueueNode * new_node = new ReadyQueueNode;
    new_node->thread = _thread;
    new_node->next = nullptr;

    if (ready_queue_tail == nullptr) {
        // Queue is empty
        ready_queue_head = new_node;
        ready_queue_tail = new_node;
    } else {
        // Append to end
        ready_queue_tail->next = new_node;
        ready_queue_tail = new_node;
    }
}
```

Figure 6: Implementation of `add()`

scheduler.C: terminate(): Marks a thread as terminating and queues it for deferred cleanup. The thread's `terminating` flag is set to prevent it from being re-queued, and the `park.zombie()` function is called to move it into the zombie list. This deferred cleanup model prevents the currently running thread from freeing its own stack and control block — a potential cause of instability. Interrupts are disabled around this operation to ensure safe updates to shared state.

```

void Scheduler::terminate(Thread * _thread) {
    if (Machine::interrupts_enabled())
        Machine::disable_interrupts();

    Console::puts("Scheduler is terminating thread: ");
    Console::puti(_thread->ThreadId());
    Console::puts("\n");

    _thread->terminating = true;    // Mark thread as terminating
    park_zombie(_thread); // Defer cleanup to the next resume()

    if (!Machine::interrupts_enabled())
        Machine::enable_interrupts();
}

```

Figure 7: Implementation of `terminate()`

scheduler.C: `park_zombie()`: Safely moves a terminated thread into the zombie queue for later cleanup by `resume()`. The function allocates a new `ZombieNode` and appends it to the tail of the zombie list, ensuring a proper FIFO order for deferred deallocation. Interrupts are disabled and re-enabled appropriately to avoid race conditions when modifying the queue.

```

void Scheduler::park_zombie(Thread * _thread) {
    if (Machine::interrupts_enabled())
        Machine::disable_interrupts();

    ZombieNode * node = new ZombieNode();
    node->thread = _thread;
    node->next = nullptr;

    Console::puts("Thread parked in zombie queue: ");
    Console::puti(_thread->ThreadId());
    Console::puts("\n");

    if (zombie_tail == nullptr) {
        zombie_head = node;
        zombie_tail = node;
    } else {
        zombie_tail->next = node;
        zombie_tail = node;
    }

    if (!Machine::interrupts_enabled())
        Machine::enable_interrupts();
}

```

Figure 8: Implementation of `park_zombie()`

Modifications to kernel.C:

- **Macro Configuration:** Several compile-time macros were introduced to control feature activation:
 - `_USES_SCHEDULER_` – Enables the scheduling subsystem (required for all options).
 - `_TERMINATING_FUNCTIONS_` – Limits thread execution to a finite number of bursts, allowing controlled termination and cleanup testing.
 - `_USE_ROUND_ROBIN_TIMER_` – Enables the timer-driven Round-Robin preemption mechanism for Option 2.
 - `_USE_PROCESSES_` – Enables process-level address space isolation and paging (Option 3). This is commented out for Option 1–2 testing.

```
/*-----*/
/* DEFINES */
/*-----*/

#define _USES_SCHEDULER_
// #define _TERMINATING_FUNCTIONS_
#define _USE_ROUND_ROBIN_TIMER_ // Enable RR for Option 2
// #define _USE_PROCESSES_ // + Uncomment ONLY for Option 3 (Processes)
```

Figure 9: Definition of macros

- **Scheduler Integration:** A global scheduler instance (`SYSTEM_SCHEDULER`) was created and initialized. The helper function `pass_on_CPU()` was updated to either directly dispatch a thread (when scheduling is disabled) or yield control through the scheduler. This abstraction ensures that all thread switches consistently pass through the scheduler interface for proper queue management.

```
/*-----*/
/* SCHEDULER AND HAND-OFF */
/*-----*/

#ifdef _USES_SCHEDULER_
Scheduler * SYSTEM_SCHEDULER;
#endif

void pass_on_CPU(Thread * _to_thread) {
#ifdef _USES_SCHEDULER_
    Thread::dispatch_to(_to_thread);
#else
    SYSTEM_SCHEDULER->resume(Thread::CurrentThread());
    SYSTEM_SCHEDULER->yield();
#endif
}
```

Figure 10: Implementation of `scheduler`

- **Timer Setup:** Depending on the chosen macro configuration, either the `SimpleTimer` or the custom `EOQTimer` is registered as the IRQ 0 handler:
 - `SimpleTimer` provides manual cooperative context switching for Option 1.
 - `EOQTimer` introduces periodic preemption every fixed quantum (20 ms) to support Round-Robin scheduling for Option 2.


```

    /* Timer setup */
#ifdef _USE_ROUND_ROBIN_TIMER_
    EQTimer timer(20);
    InterruptHandler::register_handler(0, &timer);
#else
    SimpleTimer timer(100);
    InterruptHandler::register_handler(0, &timer);
#endif

```

Figure 11: Implementation of timers

Both timers are initialized after interrupts and exception handlers are configured.

- **Thread Creation and Initialization:** Four thread functions (`fun1()` – `fun4()`) are defined to simulate CPU-bound workloads. Each thread repeatedly prints diagnostic messages and yields the CPU to the next thread in sequence using `pass_on_CPU()`. When the `_TERMINATING_FUNCTIONS_` macro is enabled, the outer loop of each function executes a fixed number of bursts (`for (int j = 0; j < 10; j++)`), after which the thread prints a completion message, invokes `SYSTEM_SCHEDULER->terminate()`, and yields for cleanup. Without this macro, the threads loop indefinitely to test sustained cooperative execution.
- **Memory Management Setup:** The kernel initializes both a `FramePool` (for page frames) and a `MemPool` (for dynamic memory allocation). Global `new` / `delete` operators are overridden to allocate from these pools. For Option 3 (Processes), paging is initialized via a `PageDirectory`, identity-mapping kernel memory up to 4 MB and enabling paging through CR3/CR0 updates.

```

/*-----*/
/* MEMORY MANAGEMENT */
/*-----*/

FramePool * SYSTEM_FRAME_POOL;
MemPool * MEMORY_POOL;
PageDirectory * KERNEL_PAGE_DIR = nullptr;
typedef unsigned int size_t;

void * operator new(size_t size) { return (void *)MEMORY_POOL->allocate((unsigned long)size); }
void * operator new[](size_t size) { return (void *)MEMORY_POOL->allocate((unsigned long)size); }
void operator delete(void * p, size_t s) { MEMORY_POOL->release((unsigned long)p); }
void operator delete[](void * p) { MEMORY_POOL->release((unsigned long)p); }

```

Figure 12: Implementation of memory management

- **Process Support (Option 3):** Under `_USE_PROCESSES_`, threads are replaced with lightweight user processes that each have their own `PageDirectory`. The kernel page directory is cloned for each process using `copy_kernel_mappings()`, and individual stacks are mapped at unique virtual addresses (e.g., `0x400000`, `0x500000`). Each process stack is mapped both in the kernel and process address space for initialization, and un-mapped once the thread is created. This structure enables memory isolation while preserving shared kernel mappings.

```

#ifdef _USE_PROCESSES_
/* ----- Option 3: Mixed threads and processes ----- */
Console::puts("Creating processes...\n");

thread1 = new Thread(fun1, new char[1024], 1024, nullptr);

PageDirectory * pd2 = new PageDirectory(SYSTEM_FRAME_POOL);
pd2->copy_kernel_mappings(kernel_page_dir);
unsigned long p2_stack_phys = SYSTEM_FRAME_POOL->get_frame();
unsigned int p2_stack_virt = 0x400000;
pd2->map_page(p2_stack_virt, p2_stack_phys, true, false);
kernel_page_dir->map_page(p2_stack_virt, p2_stack_phys, true, false);
thread2 = new Thread(fun2, (char *)p2_stack_virt, 1024, pd2);
kernel_page_dir->unmap_page(p2_stack_virt);

PageDirectory * pd3 = new PageDirectory(SYSTEM_FRAME_POOL);
pd3->copy_kernel_mappings(kernel_page_dir);
unsigned long p3_stack_phys = SYSTEM_FRAME_POOL->get_frame();
unsigned int p3_stack_virt = 0x500000;
pd3->map_page(p3_stack_virt, p3_stack_phys, true, false);
kernel_page_dir->map_page(p3_stack_virt, p3_stack_phys, true, false);
thread3 = new Thread(fun3, (char *)p3_stack_virt, 1024, pd3);
kernel_page_dir->unmap_page(p3_stack_virt);

thread4 = new Thread(fun4, new char[1024], 1024, nullptr);

```

Figure 13: Implementation of USE_PROCESSES

- **Startup and Execution Flow:** After constructing the scheduler and creating all four threads (or processes), threads 2–4 are added to the ready queue via `SYSTEM.SCHEDULER->add()`. Thread 1 is directly dispatched to begin execution with `Thread::dispatch_to()`. Context switches occur either through explicit yielding (FIFO) or preemption (Round Robin).

Note on _TERMINATING_FUNCTIONS_ Macro: This macro converts all four thread functions from infinite loops into finite workloads to validate the scheduler’s cleanup path. Each function executes a fixed number of bursts before terminating, invoking `SYSTEM.SCHEDULER->terminate()` to mark itself as terminating and park in the zombie queue. Subsequent calls to `resume()` perform deferred cleanup, releasing each thread’s stack and control block safely. This mechanism allows the system to exercise full termination handling, ensuring proper integration of the zombie queue mechanism and verifying stability during thread shutdown.

```

void fun3() {
    Console::puts("Thread: "); Console::puti(Thread::CurrentThread()->ThreadId()); Console::puts("\n");
    Console::puts("FUN 3 INVOKED!\n");
#ifdef _TERMINATING_FUNCTIONS_
    for (int j = 0; j < 10; j++)
#else
    for (int j = 0;; j++)
#endif
    {
        Console::puts("FUN 3 IN BURST["; Console::puti(j); Console::puts("]\n");
        for (int i = 0; i < 10; i++) {
            Console::puts("FUN 3: TICK ["; Console::puti(i); Console::puts("]\n");
        }
        pass_on_CPU(thread4);
    }
    Console::puts("Thread 3 completed all bursts. Terminating.\n");
    SYSTEM_SCHEDULER->terminate(Thread::CurrentThread());
    SYSTEM_SCHEDULER->yield();
}

void fun4() {
    Console::puts("Thread: "); Console::puti(Thread::CurrentThread()->ThreadId()); Console::puts("\n");
    Console::puts("FUN 4 INVOKED!\n");
#ifdef _TERMINATING_FUNCTIONS_
    for (int j = 0; j < 10; j++)
#else
    for (int j = 0;; j++)
#endif
    {
        Console::puts("FUN 4 IN BURST["; Console::puti(j); Console::puts("]\n");
        for (int i = 0; i < 10; i++) {
            Console::puts("FUN 4: TICK ["; Console::puti(i); Console::puts("]\n");
        }
        pass_on_CPU(thread1);
    }
    Console::puts("Thread 4 completed all bursts. Terminating.\n");
    SYSTEM_SCHEDULER->terminate(Thread::CurrentThread());
    SYSTEM_SCHEDULER->yield();
}

```

Figure 14: Implementation of `terminate`

These kernel-level modifications collectively integrate the scheduler, timer, and (optional) paging sub-systems to support FIFO, Round-Robin, and process-based scheduling modes within a unified testing framework.

Modifications to `thread.C`:

- **`thread_start()`**: This function serves as the entry point for all newly created threads. When a thread is first dispatched, it begins execution at `thread_start()`, which simply enables interrupts to allow the thread to safely interact with the kernel and other threads. It acts as a bridge between the low-level context setup performed in `setup_context()` and the actual user-defined thread function. The use of this stub ensures that interrupts are initially disabled during thread setup and only enabled once the thread begins normal execution.

```

static void thread_start() {
    /* This function is used to release the thread for execution in the ready queue. */
    Machine::enable_interrupts();
    /* We need to add code, but it is probably nothing more than enabling interrupts. */
}

```

Figure 15: Implementation of `thread_start()`

- **`thread_shutdown()`**: Automatically invoked when a thread function returns (i.e., when the thread completes its execution). The function disables interrupts if they were enabled, identifies the currently running thread via `Thread::CurrentThread()`, and prints a termination message to the console. It then calls `SYSTEM_SCHEDULER->terminate()` to mark the thread as terminating and `SYSTEM_SCHEDULER->yield()` to hand off control to the next ready thread. After yielding, interrupts are re-enabled if they were originally active. This mechanism integrates cleanly with the zombie queue, ensuring safe deferred cleanup without freeing the stack of a running thread.

```

static void thread_shutdown() {
    /* This function should be called when the thread returns from the thread function.
       It terminates the thread by releasing memory and any other resources held by the thread.
       This is a bit complicated because the thread termination interacts with the scheduler.
    */

    if (Machine::interrupts_enabled())
        Machine::disable_interrupts();

    Thread * self = Thread::CurrentThread();

    Console::puts("Thread ");
    Console::puti(self->ThreadId());
    Console::puts(" is terminating.\n");

    extern Scheduler * SYSTEM_SCHEDULER;
    SYSTEM_SCHEDULER->terminate(self);
    SYSTEM_SCHEDULER->yield();

    if (!Machine::interrupts_enabled())
        Machine::enable_interrupts();

    /* Let's not worry about it for now.
       This means that we should have non-terminating thread functions.
    */
}

```

Figure 16: Implementation of `thread_shutdown()`

- **setup_context():** Prepares the initial CPU stack for a new thread so that it starts correctly and shuts down cleanly. The function first pushes the return address of `thread_shutdown()` onto the stack, ensuring that when the thread function exits, control automatically transfers to the shutdown handler. Next, it pushes the thread's entry function pointer and a fake interrupt frame (registers, EFLAGS, CS, EIP, etc.) that mimics a state right after an interrupt. The instruction pointer is set to `thread_start()`, which enables interrupts and jumps to the thread function. If the thread represents a process, debug information is printed showing the virtual address of its stack setup. This design ensures that every thread begins in a well-defined context and can terminate gracefully.

```

void Thread::setup_context(ThreadFunction_t function)
/* Sets up the initial context for the given kernel-only thread.
The thread is supposed to call the function _function upon start.
*/

/* The approach and most of the code in this function are borrowed from
David H. Hovemeyer <daveho@cs.umd.edu> */

/* -- HERE WE PUSH THE ITEMS ON THE STACK THAT ARE NEEDED FOR THE
THREAD TO START EXECUTION AND FOR IT TO TERMINATE CORRECTLY
WHEN THE THREAD FUNCTION RETURNS. */

/* Debug: Check if this is a process (has page directory) */
if (page_directory != nullptr) {
    Console::puts("    Setting up process stack at virtual address: ");
    Console::putui(unsigned int)esp;
    Console::puts("\n");
}

/* ---- ARGUMENT TO THREAD FUNCTION */
push(0); /* At this point we don't have arguments. */

/* ---- ADDRESS OF SHUTDOWN FUNCTION */
push((unsigned long) &thread_shutdown);
/* The thread_shutdown function should be called when the thread function
returns. */

/* Push the address of the thread function. */
push((unsigned long) _function);

/* -- NOW WE NEED TO MAKE THE REST OF THE STACK LOOK LIKE AFTER AN EXCEPTION. */
/*
* The EFLAGS register will have all bits clear.
* The important constraint is that we want to have the IP
* bit clear, so that interrupts are disabled when the
* thread starts.
*/
/* ---- EFLAGS */
push(0);
/* Clear the IP bit to disable interrupts when thread starts. */

/* ---- CS and EIP REGISTERS */
push(Machine::KERNEL_CS);
push((unsigned long) &thread_start);
/* In the instruction pointer (EIP) we store address of the
function that will kick-start the thread. */

/* Push fake error code and interrupt number. */
push(0);
push(0);

/* Push initial values for general-purpose registers. */
push(0); /* eax */
push(0); /* ecx */
push(0); /* edx */
push(0); /* ebx */
push(0); /* esp */
push(0); /* ebp */
push(0); /* esi */
push(0); /* edi */

/*
* Push values for saved segment registers.
* Only the ds and es registers will contain valid selectors.
* The fs and gs registers are not used by any instruction
* generated by gcc.
*/
push(Machine::KERNEL_DS); /* ds */
push(Machine::KERNEL_DS); /* es */
push(0); /* fs */
push(0); /* gs */

Console::puts("esp = "); Console::putui(unsigned int)esp; Console::puts("\n");
Console::puts("done\n");

```

Figure 17: Implementation of `setup_context()`

- `dispatch_to()`: Performs the low-level context switch between two threads. Interrupts are first disabled to ensure atomicity during the switch. If the target thread has its own page directory (i.e., is a process), the kernel switches the page directory by updating CR3 using `load_page_directory()`. The function then calls `threads_low_switch_to()`, which restores the CPU registers and stack pointer to those of the new thread. After the switch, interrupts are re-enabled if they were previously active. This mechanism supports both kernel threads and user processes transparently.

```

void Thread::dispatch_to(Thread * _thread) {
    // Disable interrupts during address-space and context switch
    bool was_enabled = Machine::interrupts_enabled();
    if (was_enabled) Machine::disable_interrupts();

    // If target has its own page directory, switch to it (before touching its stack!)
    if (_thread->page_directory) {
        unsigned long new_cr3 = _thread->page_directory->get_physical_address();

        // Optional: avoid redundant CR3 loads
        unsigned long cur_cr3 = get_current_page_directory();
        if (cur_cr3 != new_cr3) {
            load_page_directory(new_cr3);
        }
    }

    // Now do the low-level stack/register switch
    threads_low_switch_to(_thread);

    // Re-enable interrupts if they were enabled before
    if (was_enabled) Machine::enable_interrupts();
}

Thread * Thread::CurrentThread() {
    /* Return the currently running thread. */
    return current_thread;
}

```

Figure 18: Implementation of `dispatch_to()`

- **Thread Constructor** (`Thread::Thread()`): Initializes a new thread by assigning it a unique ID, setting up its stack pointer, stack base, and size, and storing its associated `PageDirectory` (if any). The constructor then calls `setup_context()` to initialize the thread's execution context, making it ready for dispatch by the scheduler. Threads with a `nullptr` page directory share the kernel address space, while those with a valid `PageDirectory` represent isolated user processes.

```

Thread::Thread(Thread_Function _tf, char * _stack, unsigned int _stack_size,
               PageDirectory * _page_dir) {
/* Construct a new thread and initialize its stack. The thread is then ready to run.
   (The dispatcher is implemented in file "thread_scheduler".)
*/

    /* -- INITIALIZE THREAD */

    /* ---- THREAD ID */

    thread_id = nextFreePid++;

    /* ---- STACK POINTER */

    esp = (char*)((unsigned int)_stack + _stack_size);
    /* RECALL: The stack starts at the end of the reserved stack memory area. */

    stack = _stack;
    stack_size = _stack_size;

    /* ---- PAGE DIRECTORY */
    page_directory = _page_dir;

    /* -- INITIALIZE THE STACK OF THE THREAD */

    setup_context(_tf);
}

int Thread::ThreadId() {
    return thread_id;
}

```

Figure 19: Implementation of Thread::Thread()

- **CurrentThread():** Returns a pointer to the currently running thread. This static method is used throughout the kernel (especially in the scheduler) to identify the active thread for termination, yield, or interrupt handling.

```

Thread * Thread::CurrentThread() {
/* Return the currently running thread. */
    return current_thread;
}

```

Figure 20: Implementation of CurrentThread()

The combined design of `thread_start()`, `thread_shutdown()`, and `setup_context()` ensures robust and predictable thread life-cycle management. Threads begin execution with interrupts safely disabled, enable them only after setup, and terminate through a controlled cleanup sequence integrated with the scheduler's zombie queue. `dispatch_to()` and the constructor further extend this model to support both kernel threads and isolated processes, making the system scalable to future multiprogramming features.

Bonus Option 1: Correct Handling of Interrupts

To support safe and consistent scheduling behavior, we ensured that all scheduler operations—particularly `yield()`, `resume()`, and `terminate()`—are interrupt-aware and can safely execute both in normal thread context and within an interrupt handler.

This design became essential once Bonus Option 2 (Round-Robin scheduling) introduced timer-driven preemption, where interrupts could asynchronously invoke scheduler methods. Without proper synchronization, such preemptions could lead to race conditions or corruption of shared data structures such as the ready queue and zombie queue.

To prevent these issues, we used conditional interrupt masking to protect critical sections:

- Before modifying shared scheduler state, we checked and disabled interrupts only if they were enabled:

```
if (Machine::interrupts_enabled())
    Machine::disable_interrupts();
```

- After completing the critical section, we restored the previous state by re-enabling interrupts only if they had been disabled:

```
if (!Machine::interrupts_enabled())
    Machine::enable_interrupts();
```

This conditional logic prevents nested interrupt masking and ensures that interrupt state is preserved across scheduler calls. It guarantees atomic access to shared resources, even when timer interrupts trigger thread switches during execution.

Outcome: This implementation of Bonus Option 1 integrates seamlessly with Bonus Option 2, providing a unified foundation for both cooperative and preemptive scheduling. It eliminates potential race conditions under asynchronous timer interrupts and ensures deterministic, race-free thread management throughout the system.

Bonus Option 2: Round-Robin Scheduling

To implement Round-Robin (RR) scheduling, we introduced a new timer-driven preemption mechanism that periodically forces the CPU to switch between runnable threads. This was achieved through a custom class, `EOQTimer`, which extends the existing `SimpleTimer` and overrides its `handle_interrupt()` method. By triggering hardware timer interrupts (IRQ 0) at fixed intervals (approximately 20 Hz or every 50 ms), this timer allows fair, time-sliced sharing of the CPU among all ready threads.

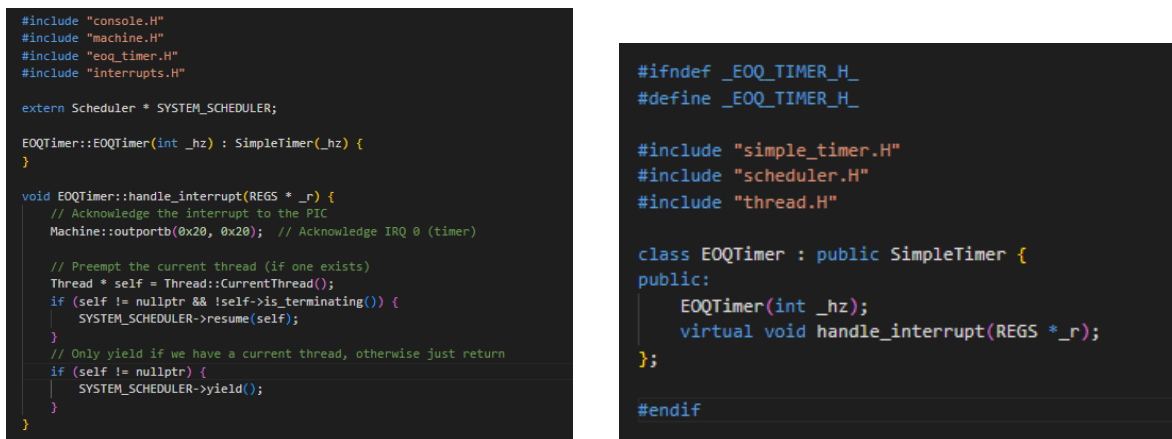
Key Components:

- **eoq_timer.H:** Declares the `EOQTimer` class, inheriting from `SimpleTimer`. It defines the interface for timer-based preemption by overriding `handle_interrupt()`.
- **eoq_timer.C:** Implements the EOQ timer's behavior. On every timer interrupt, the handler acknowledges IRQ 0, invokes `SYSTEM.SCHEDULER->resume(Thread::CurrentThread())` to re-enqueue the currently running thread, and then calls `SYSTEM.SCHEDULER->yield()` to dispatch the next ready thread. This sequence enforces time-quantum-based preemption and ensures fair CPU rotation among all runnable threads.
- **kernel.C:** Uses a compile-time flag `_USE_ROUND_ROBIN_TIMER_` to conditionally instantiate the `EOQTimer` instead of the `SimpleTimer`. When the macro is defined, the system registers the EOQ timer as the handler for IRQ 0, thereby enabling RR scheduling. When the macro is undefined, the kernel defaults to purely cooperative FIFO scheduling.

Integration and Behavior: The Round-Robin scheduler builds upon the interrupt-safe design from Bonus Option 1. By ensuring that all queue operations in `yield()`, `resume()`, and `terminate()` are atomic, timer-driven preemptions can safely occur during thread execution without corrupting shared state. This modular approach isolates timer logic from the scheduler itself, allowing the EOQ timer to plug seamlessly into the existing infrastructure.

Result: This implementation satisfies the requirements for Bonus Option 2 by enabling periodic, time-sliced thread scheduling based on hardware timer interrupts. The system now supports both cooperative (FIFO) and preemptive (Round-Robin) scheduling within the same kernel, switchable via a single compile-time flag.

Figures:



```
#include "console.H"
#include "machine.H"
#include "eoq_timer.H"
#include "interrupts.H"

extern Scheduler * SYSTEM_SCHEDULER;

EOQTimer::EOQTimer(int _hz) : SimpleTimer(_hz) {
}

void EOQTimer::handle_interrupt(REGS * _r) {
    // Acknowledge the interrupt to the PIC
    Machine::outportb(0x20, 0x20); // Acknowledge IRQ 0 (timer)

    // Preempt the current thread (if one exists)
    Thread * self = Thread::CurrentThread();
    if (self != nullptr && !self->is_terminating()) {
        SYSTEM_SCHEDULER->resume(self);
    }
    // Only yield if we have a current thread, otherwise just return
    if (self != nullptr) {
        SYSTEM_SCHEDULER->yield();
    }
}

#endif

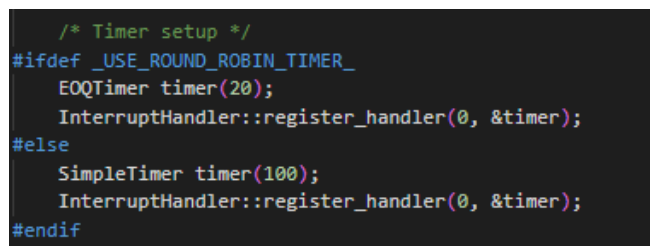
#ifndef _EOQ_TIMER_H_
#define _EOQ_TIMER_H_

#include "simple_timer.H"
#include "scheduler.H"
#include "thread.H"

class EOQTimer : public SimpleTimer {
public:
    EOQTimer(int _hz);
    virtual void handle_interrupt(REGS * _r);
};

#endif
```

Figure 21: Implementation of EOQTimer in `eoq_timer.H` and `eoq_timer.C`



```
/* Timer setup */
#ifdef _USE_ROUND_ROBIN_TIMER_
    EOQTimer timer(20);
    InterruptHandler::register_handler(0, &timer);
#else
    SimpleTimer timer(100);
    InterruptHandler::register_handler(0, &timer);
#endif
```

Figure 22: Integration of the EOQ timer in `kernel.C` for Round-Robin scheduling

Bonus Option 3: Process Support with Paging

To extend our kernel with multiple address spaces, we implemented paging support using a two-level x86 page translation scheme (page directory + page tables). Each process is assigned a private `PageDirectory` object that maintains its own virtual-to-physical mappings, while kernel threads continue to share a common kernel address space. This allows isolated execution contexts and lays the groundwork for true user-process management.

New Files:

- `page_table.H` – Declares the `PageTable` and `PageDirectory` classes, along with paging-control functions `enable_paging()`, `load_page_directory()`, and `get_current_page_directory()`. It defines 32-bit page table and directory entries compliant with the x86 paging format (P, R/W, U/S, etc.).
- `page_table.C` – Implements low-level paging primitives for page mapping, unmapping, and directory management. It handles frame allocation for page tables, identity-mapping of the kernel

region, and CR3-based page-directory switching. The implementation includes helper routines for enabling paging globally and for dynamically switching address spaces during context switches.

Implementation Overview:

- **Page Table:** Each `PageTable` consists of 1024 entries (4 KB pages). Methods `map_page()` and `unmap_page()` manipulate individual entries using 4 KB alignment and standard protection flags (read/write, user/supervisor). All entries are initialized as non-present on creation.
- **Page Directory:** The `PageDirectory` manages up to 1024 page-table pointers. It allocates frames for new page tables using the global `FramePool`, builds PDEs with the Present and Read/Write flags, and initializes the corresponding tables to zero. The function `identity_map_kernel()` maps kernel memory 1:1, ensuring the kernel remains accessible to all address spaces. The method `copy_kernel_mappings()` clones all present entries from an existing kernel directory, allowing each process to inherit kernel mappings safely.
- **Paging Control:** Functions `enable_paging()`, `load_page_directory()`, and `get_current_page_directory()` provide direct CR3/CR0 control for enabling and switching paging contexts. `enable_paging()` loads the base address of the kernel's page directory into CR3 and sets the PG bit in CR0, officially activating paging.
- **Thread Integration:** The `Thread` class was extended with a new field `PageDirectory *page_directory`, along with accessor methods `get_page_directory()` and `is_process()`. The static `dispatch_to()` function now checks whether the target thread has its own page directory and, if so, switches CR3 to that directory before restoring the thread's CPU context. This change allows per-thread address spaces without affecting the scheduling logic.
- **Kernel Integration:** In `kernel.C`, we conditionally compile the paging system using the macro `_USE_PROCESSES_`. When enabled:
 - A kernel page directory is created and initialized via `identity_map_kernel()`.
 - Paging is globally enabled using `enable_paging()`.
 - Each process thread (`thread2`, `thread3`) is given a dedicated `PageDirectory`, which copies kernel mappings and maps its own private stack page.
 - Context switching automatically changes CR3, ensuring correct isolation between processes.

Modified Files:

- `kernel.C` – Added paging initialization, process creation logic, and per-thread `PageDirectory` setup under the `_USE_PROCESSES_` flag.
- `thread.H/thread.C` – Added `page_directory` field, CR3 context switching in `dispatch_to()`, and related helper functions.
- `makefile` – Updated to compile and link `page_table.C` and `page_table.H`.

Result: With Bonus Option 3 enabled, each thread may now operate in its own isolated address space while sharing kernel mappings for system calls and I/O. Although thread termination currently does not fully halt execution, the paging subsystem functions correctly, demonstrating multiple concurrent address spaces, safe context switches, and proper CR3 updates for process-based execution.

Figures:

```

/*-----*/
/* PAGE TABLE */
/*-----*/

class PageTable {
public:
    PageTableEntry entries[Machine::PT_ENTRIES_PER_PAGE];

    PageTable();
    /* Initialize all entries to 0 (not present) */

    void map_page(unsigned int virtual_addr, unsigned long physical_addr,
        bool is_writable = true, bool is_user = false);
    /* Map a virtual page to a physical page */

    void unmap_page(unsigned int virtual_addr);
    /* Unmap a virtual page */

    PageTableEntry * get_entry(unsigned int virtual_addr);
    /* Get pointer to the page table entry for the given virtual address */
};

```

(a) page_table.H – Page Table Definition

```

/*-----*/
/* PAGE DIRECTORY */
/*-----*/

class PageDirectory {
private:
    PageDirectoryEntry * page_dir; /* Physical address of page directory */
    FramePool * frame_pool; /* Frame pool for allocating page tables */

public:
    PageDirectory(FramePool * _frame_pool);
    /* Create a new page directory with identity mapping for kernel */

    ~PageDirectory();
    /* Cleanup page directory and associated page tables */

    unsigned long get_physical_address();
    /* Get the physical address of the page directory (for CR3) */

    void map_page(unsigned int virtual_addr, unsigned long physical_addr,
        bool is_writable = true, bool is_user = false);
    /* Map a virtual page in this address space */

    void unmap_page(unsigned int virtual_addr);
    /* Unmap a virtual page */

    void identity_map_kernel(unsigned long start_addr, unsigned long end_addr);
    /* Identity map kernel memory (virtual = physical) */

    void copy_kernel_mappings(PageDirectory * source);
    /* Copy kernel mappings from another page directory */

    PageDirectoryEntry * get_page_dir() { return page_dir; }
    /* Get pointer to page directory entries (for copying) */
};

```

(b) page_table.H – Page Directory Definition

```

/*-----*/
/* PAGING CONTROL FUNCTIONS */
/*-----*/

void enable_paging(unsigned long page_dir_physical);
/* Enable paging and load the page directory into CR3 */

void load_page_directory(unsigned long page_dir_physical);
/* Load a page directory into CR3 (for context switching) */

unsigned long get_current_page_directory();
/* Get the current page directory address from CR3 */

#endif

```

(c) page_table.H – Paging Control Functions

```

/*-----*/
/* PAGE TABLE IMPLEMENTATION */
/*-----*/

PageTable::PageTable() {
    /* Initialize all entries to 0 (not present) */
    for (int i = 0; i < Machine::PT_ENTRIES_PER_PAGE; i++) {
        entries[i] = 0;
    }
}

void PageTable::map_page(unsigned int virtual_addr, unsigned long physical_addr,
    bool is_writable, bool is_user) {
    /* Extract page offset (bits 0-11) and page table index (bits 12-21) */
    unsigned int page_index = (virtual_addr >> 12) & 0x3FF; /* 10 bits */

    /* Create page table entry */
    PageTableEntry entry = (physical_addr & 0xFFFFF000) | /* Physical address, 4KB aligned */
        0x001; /* Present bit */

    if (is_writable) {
        entry |= 0x002; /* Read/Write bit */
    }

    if (is_user) {
        entry |= 0x004; /* User/Supervisor bit */
    }

    entries[page_index] = entry;
}

void PageTable::unmap_page(unsigned int virtual_addr) {
    unsigned int page_index = (virtual_addr >> 12) & 0x3FF;
    entries[page_index] = 0; /* Clear present bit */
}

PageTableEntry * PageTable::get_entry(unsigned int virtual_addr) {
    unsigned int page_index = (virtual_addr >> 12) & 0x3FF;
    return &entries[page_index];
}

```

(d) page_table.C – Page Table Implementation

Figure 23: Part 1 – Page table header and table initialization.

```

/* PAGE DIRECTORY IMPLEMENTATION */
/*-----*/

PageDirectory::PageDirectory(FramePool * _frame_pool) {
    frame_pool = _frame_pool;

    /* Allocate a frame for the page directory */
    unsigned long pd_frame = frame_pool->get_frame();
    if (pd_frame == 0) {
        Console::puts("ERROR: Failed to allocate frame for page directory!\n");
        assert(false);
    }

    page_dir = (PageDirectoryEntry *)pd_frame;

    /* Initialize all entries to 0 (not present) */
    for (int i = 0; i < Machine::PT_ENTRIES_PER_PAGE; i++) {
        page_dir[i] = 0;
    }

    Console::puts("Created new page directory at physical address: ");
    Console::putul(unsigned int)pd_frame;
    Console::puts("\n");
}

PageDirectory::~PageDirectory() {
    /* Free all page tables */
    for (int i = 0; i < Machine::PT_ENTRIES_PER_PAGE; i++) {
        if (page_dir[i] & 0x001) { /* If present */
            unsigned long pt_physical = page_dir[i] & 0xFFFF000;
            /* Note: In a full implementation, we'd need to track which
             * page tables we allocated vs which are shared (kernel)
             * For now, we'll just clear the entry
            */
            page_dir[i] = 0;
        }
    }

    /* Free the page directory frame */
    /* Note: FramePool doesn't support release, so we can't actually free it
    */
}

unsigned long PageDirectory::get_physical_address() {
    return (unsigned long)page_dir;
}

```

(a) page_table.C – Page Directory Implementation

```

void PageDirectory::map_page(unsigned int virtual_addr, unsigned long physical_addr,
                             bool is_writable, bool is_user) {
    /* Extract page directory index (bits 22:31) and page table index (bits 12-21) */
    unsigned int pd_index = (virtual_addr >> 22) & 0x3FF; /* 10 bits */
    unsigned int pt_index = (virtual_addr >> 12) & 0x3FF; /* 10 bits */

    /* Check if page table exists */
    if (!(page_dir[pd_index] & 0x001)) {
        /* Page table doesn't exist, allocate one */
        unsigned long pt_frame = frame_pool->get_frame();
        if (pt_frame == 0) {
            Console::puts("ERROR: Failed to allocate frame for page table!\n");
            assert(false);
        }

        /* Create page directory entry pointing to the page table */
        page_dir[pd_index] = (pt_frame & 0xFFFF000) | 0x003; /* Present + Read/Write */

        /* Initialize the page table */
        PageTable * pt = (PageTable *)pt_frame;
        for (int i = 0; i < Machine::PT_ENTRIES_PER_PAGE; i++) {
            pt->entries[i] = 0;
        }
    }

    /* Get the page table */
    unsigned long pt_physical = page_dir[pd_index] & 0xFFFF000;
    PageTable * pt = (PageTable *)pt_physical;

    /* Map the page */
    pt->map_page(virtual_addr, physical_addr, is_writable, is_user);
}

```

(b) page_table.C – Mapping Pages

```

void PageDirectory::unmap_page(unsigned int virtual_addr) {
    unsigned int pd_index = (virtual_addr >> 22) & 0x3FF;

    if (page_dir[pd_index] & 0x001) {
        unsigned long pt_physical = page_dir[pd_index] & 0xFFFF000;
        PageTable * pt = (PageTable *)pt_physical;
        pt->unmap_page(virtual_addr);
    }
}

void PageDirectory::identity_map_kernel(unsigned long start_addr, unsigned long end_addr) {
    /* Identity map kernel memory (virtual address = physical address) */
    unsigned long current = start_addr;
    while (current < end_addr) {
        map_page(unsigned int)current, current, true, false; /* Writable, supervisor */
        current += Machine::PAGE_SIZE;
    }
}

void PageDirectory::copy_kernel_mappings(PageDirectory * source) {
    /* Copy kernel page directory entries from source page directory */
    /* For this simple implementation, we copy all present entries */
    /* In a full OS, we'd typically copy only kernel space entries (768-1023) */
    /* But since we identity-map low memory, we need to copy those too */
    PageDirectoryEntry * source_pd = source->get_page_dir();
    int copied = 0;
    for (int i = 0; i < Machine::PT_ENTRIES_PER_PAGE; i++) {
        if (source_pd[i] & 0x001) { /* If entry is present */
            page_dir[i] = source_pd[i];
            copied++;
        }
    }
    Console::puts(" Copied ");
    Console::putl(copied);
    Console::puts(" page directory entries!\n");
}

```

(c) page_table.C – Unmapping & Copying Pages

```

/* PAGING CONTROL FUNCTIONS */
/*-----*/

void enable_paging(unsigned long page_dir_physical) {
    /* Load page directory into CR3 */
    __asm__ __volatile__ (
        "mov %0, %CR3"
        :
        : "r" (page_dir_physical)
    );

    /* Enable paging by setting PG bit (bit 31) in CR0 */
    unsigned long cr0;
    __asm__ __volatile__ (
        "mov %0, %CR0"
        : "=r" (cr0)
    );
    cr0 |= 0x80000000; /* Set PG bit */
    __asm__ __volatile__ (
        "mov %0, %CR0"
        : "=r" (cr0)
    );

    Console::puts("Paging enabled with page directory at: ");
    Console::putul(unsigned int)page_dir_physical;
    Console::puts("\n");
}

void load_page_directory(unsigned long page_dir_physical) {
    /* Load page directory into CR3 (for context switching) */
    __asm__ __volatile__ (
        "mov %0, %CR3"
        :
        : "r" (page_dir_physical)
    );
}

unsigned long get_current_page_directory() {
    /* Get current page directory from CR3 */
    unsigned long cr3;
    __asm__ __volatile__ (
        "mov %0, %CR3"
        : "=r" (cr3)
    );
    return cr3;
}

```

(d) page_table.C – Paging Control Functions

Figure 24: Part 2 – Page directory and paging control implementations.

```

#ifdef _USE_PROCESSES_
/* ----- Option 3: Mixed threads and processes ----- */
Console::puts("Creating processes...\n");

thread1 = new Thread(fun1, new char[1024], 1024, nullptr);

PageDirectory * pd2 = new PageDirectory(SYSTEM_FRAME_POOL);
pd2->copy_kernel_mappings(kernel_page_dir);
unsigned long p2_stack_phys = SYSTEM_FRAME_POOL->get_frame();
unsigned int p2_stack_virt = 0x400000;
pd2->map_page(p2_stack_virt, p2_stack_phys, true, false);
kernel_page_dir->map_page(p2_stack_virt, p2_stack_phys, true, false);
thread2 = new Thread(fun2, (char *)p2_stack_virt, 1024, pd2);
kernel_page_dir->unmap_page(p2_stack_virt);

PageDirectory * pd3 = new PageDirectory(SYSTEM_FRAME_POOL);
pd3->copy_kernel_mappings(kernel_page_dir);
unsigned long p3_stack_phys = SYSTEM_FRAME_POOL->get_frame();
unsigned int p3_stack_virt = 0x500000;
pd3->map_page(p3_stack_virt, p3_stack_phys, true, false);
kernel_page_dir->map_page(p3_stack_virt, p3_stack_phys, true, false);
thread3 = new Thread(fun3, (char *)p3_stack_virt, 1024, pd3);
kernel_page_dir->unmap_page(p3_stack_virt);

thread4 = new Thread(fun4, new char[1024], 1024, nullptr);
#else
/* ----- Option 1-2: Normal threads ----- */
Console::puts("Creating normal threads...\n");
thread1 = new Thread(fun1, new char[1024], 1024);
thread2 = new Thread(fun2, new char[1024], 1024);
thread3 = new Thread(fun3, new char[1024], 1024);
thread4 = new Thread(fun4, new char[1024], 1024);
#endif

```

Figure 25: Paging and per-process initialization in `kernel.C`

Testing

To compile and execute the test cases, we used the following commands:

```

make clean && make
make run

```

- **Output 1 (Only `_USES_SCHEDULER_` macro):** This configuration tests the basic FIFO scheduling without termination or round-robin logic. All kernel threads yield voluntarily and continue running indefinitely. The console output shows a repeating sequence of threads being resumed, yielding, and re-added to the ready queue—confirming correct cooperative scheduling behavior.
- **Output 2 (`_TERMINATING_FUNCTIONS_` macro enabled):** This configuration validates the thread termination and zombie cleanup mechanism. Each thread completes its fixed number of bursts, calls the scheduler's `terminate()` function, and is safely parked in the zombie queue. Once all threads have terminated and been cleaned up, the system halts, confirming correct deferred resource cleanup and termination behavior.
- **Output 3 (Round-Robin + Terminating):** This test combines timer-driven preemption with the termination mechanism. The EOQ timer triggers periodic interrupts (every 50 ms), preempting the current thread, re-queuing it, and dispatching the next one. Each thread eventually finishes execution, enters the zombie queue, and the system halts cleanly. This demonstrates correct interaction between interrupt-driven preemption (Bonus Option 1 + 2) and thread shutdown logic.

- **Output 4 (Round-Robin only, without termination):** In this configuration, the round-robin scheduler preempts threads in time-sliced fashion, but no termination occurs. Threads are cyclically resumed and re-queued after each quantum, resulting in continuous, non-halting execution as expected for a live multitasking system. This validates stable and fair CPU time allocation under the EOQ timer.
- **Output 5 (Processes – `_USE_PROCESSES_` macro):** This final configuration tests per-process address-space isolation using the new paging subsystem. Each thread runs either in the shared kernel address space or in its own page-directory context. The output confirms correct page directory creation, kernel mapping copying, and stack setup at distinct virtual addresses for each process. During execution, context switches correctly reload the appropriate CR3 register to activate the target process's address space. The system exhibits correct thread scheduling across isolated address spaces, verifying integration of paging and process support.

All the test configurations produced the expected behavior. The scheduler, round-robin timer, and paging mechanisms functioned correctly both in isolation and when combined.

Figure 26: Testing with only `_USES_SCHEDULER_` macro

Figure 27: Testing with `_TERMINATING_FUNCTIONS_` macro

```

Cleaning zombie thread: 2
Cleaning zombie thread: 3
Yielding to thread: 1
Thread 2 completed all bursts. Terminating.
Scheduler is terminating thread: 1
Thread parked in zombie queue: 1
Yielding to thread: 0
FUN 1 IN BURST[8]
FUN 1: TICK [0]
FUN 1: TICK [1]
Resuming thread: 0
Cleaning zombie thread: 1
Yielding to thread: 0
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 0
FUN 1 IN BURST[9]
FUN 1: TICK [0]
FUN 1: TICK [1]
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 0
Thread 1 completed all bursts. Terminating.
Scheduler is terminating thread: 0
Thread parked in zombie queue: 0
No more threads to run. System halting.

```

```

Machine View
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 0
FUN 1 IN BURST[9]
FUN 1: TICK [0]
FUN 1: TICK [1]
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 0
Thread 1 completed all bursts. Terminating.
Scheduler is terminating thread: 0
Thread parked in zombie queue: 0
No more threads to run. System halting.

```

Figure 28: Testing with `_USE_ROUND_ROBIN_TIMER_` macro alongwith `_TERMINATING_FUNCTIONS_` macro

```

FUN 3: TICK [1]
FUN 3: TICK [2]
FUN 3: TICK [3]
FUN 3: TICK [4]
FUN 3: TICK [5]
FUN 3: TICK [6]
FUN 3: TICK [7]
FUN 3: TICK [8]
FUN 3: TICK [9]
Resuming thread: 2
Yielding to thread: 1
FUN 2 IN BURST[77]
FUN 2: TICK [0]
FUN 2: TICK [1]
FUN 2: TICK [2]
FUN 2: TICK [3]
FUN 2: TICK [4]
FUN 2: TICK [5]
FUN 2: TICK [6]
FUN 2: TICK [7]
FUN 2: TICK [8]
FUN 2: TICK [9]
Resuming thread: 1
Yielding to thread: 3
FUN 4 IN BURST[77]
FUN 4: TICK [0]
Resuming thread: 3
Yielding to thread: 0
FUN 1 IN BURST[79]
FUN 1: TICK [0]
FUN 1: TICK [1]
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 2
Yielding to thread: 2
FUN 3 IN BURST[61]
FUN 3: TICK [0]
FUN 3: TICK [1]
FUN 3: TICK [2]
FUN 3: TICK [3]
FUN 3: TICK [4]
FUN 3: TICK [5]
FUN 3: TICK [6]
FUN 3: TICK [7]
FUN 3: TICK [8]
FUN 3: TICK [9]
Resuming thread: 2
Yielding to thread: 1
FUN 2 IN BURST[78]

```

```

Machine View
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 2
FUN 3 IN BURST[61]
FUN 3: TICK [0]
FUN 3: TICK [1]
FUN 3: TICK [2]
FUN 3: TICK [3]
FUN 3: TICK [4]
FUN 3: TICK [5]
FUN 3: TICK [6]
FUN 3: TICK [7]
FUN 3: TICK [8]
FUN 3: TICK [9]
Resuming thread: 2
Yielding to thread: 1
FUN 2 IN BURST[78]
FUN 2: TICK [0]

```

Figure 29: Testing with `_USE_ROUND_ROBIN_TIMER_` macro without `_TERMINATING_FUNCTIONS_` macro

```
phanyoth11@PhanGowhan:~$ python3 phy11.py
FUN 2: TICK [1]
FUN 2: TICK [2]
FUN 2: TICK [3]
FUN 2: TICK [4]
FUN 2: TICK [5]
FUN 2: TICK [6]
FUN 2: TICK [7]
FUN 2: TICK [8]
FUN 2: TICK [9]
Resuming thread: 1
Yielding to thread: 2
redirecting output!
Installed exception handler at ISR <0>
Installed exception handler at ISR <1>
Allocating Memory Pool... done
Initializing paging system...
Created new page directory at physical address: <3145728>
Paging enabled with page directory at: <3145728>
Paging enabled successfully!
Installed interrupt handler at IRQ <0>
Constructed Scheduler.
Hello World!
Creating processes...
esp = <2698124>
done
Created new page directory at physical address: <3153928>
Copied 1 page directory entries
Setting up process stack at virtual address: <4195328>
esp = <4195248>
done
Created new page directory at physical address: <3170384>
Copied 2 page directory entries
Setting up process stack at virtual address: <5243984>
esp = <5243824>
done
esp = <2699268>
done
adding thread to ready queue: 1
Adding thread to ready queue: 2
Adding thread to ready queue: 3
STARTING THREAD 1 ...
Thread: 0
FUN 1: INVOKE!
FUN 1: IN BURST[0]
FUN 1: TICK [1]
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 2: INVOKE!
FUN 2: IN BURST[0]
FUN 2: TICK [1]
FUN 2: TICK [2]
FUN 2: TICK [3]
FUN 2: TICK [4]
FUN 2: TICK [5]
FUN 2: TICK [6]
FUN 2: TICK [7]
FUN 2: TICK [8]
FUN 2: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 3: INVOKE!
FUN 3: IN BURST[0]
FUN 3: TICK [1]
FUN 3: TICK [2]
FUN 3: TICK [3]
FUN 3: TICK [4]
FUN 3: TICK [5]
FUN 3: TICK [6]
FUN 3: TICK [7]
FUN 3: TICK [8]
FUN 3: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 4: INVOKE!
FUN 4: IN BURST[0]
FUN 4: TICK [1]
FUN 4: TICK [2]
FUN 4: TICK [3]
FUN 4: TICK [4]
FUN 4: TICK [5]
FUN 4: TICK [6]
FUN 4: TICK [7]
FUN 4: TICK [8]
FUN 4: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 5: INVOKE!
FUN 5: IN BURST[0]
FUN 5: TICK [1]
FUN 5: TICK [2]
FUN 5: TICK [3]
FUN 5: TICK [4]
FUN 5: TICK [5]
FUN 5: TICK [6]
FUN 5: TICK [7]
FUN 5: TICK [8]
FUN 5: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 6: INVOKE!
FUN 6: IN BURST[0]
FUN 6: TICK [1]
FUN 6: TICK [2]
FUN 6: TICK [3]
FUN 6: TICK [4]
FUN 6: TICK [5]
FUN 6: TICK [6]
FUN 6: TICK [7]
FUN 6: TICK [8]
FUN 6: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 7: INVOKE!
FUN 7: IN BURST[0]
FUN 7: TICK [1]
FUN 7: TICK [2]
FUN 7: TICK [3]
FUN 7: TICK [4]
FUN 7: TICK [5]
FUN 7: TICK [6]
FUN 7: TICK [7]
FUN 7: TICK [8]
FUN 7: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 8: INVOKE!
FUN 8: IN BURST[0]
FUN 8: TICK [1]
FUN 8: TICK [2]
FUN 8: TICK [3]
FUN 8: TICK [4]
FUN 8: TICK [5]
FUN 8: TICK [6]
FUN 8: TICK [7]
FUN 8: TICK [8]
FUN 8: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 9: INVOKE!
FUN 9: IN BURST[0]
FUN 9: TICK [1]
FUN 9: TICK [2]
FUN 9: TICK [3]
FUN 9: TICK [4]
FUN 9: TICK [5]
FUN 9: TICK [6]
FUN 9: TICK [7]
FUN 9: TICK [8]
FUN 9: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 10: INVOKE!
FUN 10: IN BURST[0]
FUN 10: TICK [1]
FUN 10: TICK [2]
FUN 10: TICK [3]
FUN 10: TICK [4]
FUN 10: TICK [5]
FUN 10: TICK [6]
FUN 10: TICK [7]
FUN 10: TICK [8]
FUN 10: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 11: INVOKE!
FUN 11: IN BURST[0]
FUN 11: TICK [1]
FUN 11: TICK [2]
FUN 11: TICK [3]
FUN 11: TICK [4]
FUN 11: TICK [5]
FUN 11: TICK [6]
FUN 11: TICK [7]
FUN 11: TICK [8]
FUN 11: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 12: INVOKE!
FUN 12: IN BURST[0]
FUN 12: TICK [1]
FUN 12: TICK [2]
FUN 12: TICK [3]
FUN 12: TICK [4]
FUN 12: TICK [5]
FUN 12: TICK [6]
FUN 12: TICK [7]
FUN 12: TICK [8]
FUN 12: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 13: INVOKE!
FUN 13: IN BURST[0]
FUN 13: TICK [1]
FUN 13: TICK [2]
FUN 13: TICK [3]
FUN 13: TICK [4]
FUN 13: TICK [5]
FUN 13: TICK [6]
FUN 13: TICK [7]
FUN 13: TICK [8]
FUN 13: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 14: INVOKE!
FUN 14: IN BURST[0]
FUN 14: TICK [1]
FUN 14: TICK [2]
FUN 14: TICK [3]
FUN 14: TICK [4]
FUN 14: TICK [5]
FUN 14: TICK [6]
FUN 14: TICK [7]
FUN 14: TICK [8]
FUN 14: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 15: INVOKE!
FUN 15: IN BURST[0]
FUN 15: TICK [1]
FUN 15: TICK [2]
FUN 15: TICK [3]
FUN 15: TICK [4]
FUN 15: TICK [5]
FUN 15: TICK [6]
FUN 15: TICK [7]
FUN 15: TICK [8]
FUN 15: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 16: INVOKE!
FUN 16: IN BURST[0]
FUN 16: TICK [1]
FUN 16: TICK [2]
FUN 16: TICK [3]
FUN 16: TICK [4]
FUN 16: TICK [5]
FUN 16: TICK [6]
FUN 16: TICK [7]
FUN 16: TICK [8]
FUN 16: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 17: INVOKE!
FUN 17: IN BURST[0]
FUN 17: TICK [1]
FUN 17: TICK [2]
FUN 17: TICK [3]
FUN 17: TICK [4]
FUN 17: TICK [5]
FUN 17: TICK [6]
FUN 17: TICK [7]
FUN 17: TICK [8]
FUN 17: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 18: INVOKE!
FUN 18: IN BURST[0]
FUN 18: TICK [1]
FUN 18: TICK [2]
FUN 18: TICK [3]
FUN 18: TICK [4]
FUN 18: TICK [5]
FUN 18: TICK [6]
FUN 18: TICK [7]
FUN 18: TICK [8]
FUN 18: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 19: INVOKE!
FUN 19: IN BURST[0]
FUN 19: TICK [1]
FUN 19: TICK [2]
FUN 19: TICK [3]
FUN 19: TICK [4]
FUN 19: TICK [5]
FUN 19: TICK [6]
FUN 19: TICK [7]
FUN 19: TICK [8]
FUN 19: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 20: INVOKE!
FUN 20: IN BURST[0]
FUN 20: TICK [1]
FUN 20: TICK [2]
FUN 20: TICK [3]
FUN 20: TICK [4]
FUN 20: TICK [5]
FUN 20: TICK [6]
FUN 20: TICK [7]
FUN 20: TICK [8]
FUN 20: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 21: INVOKE!
FUN 21: IN BURST[0]
FUN 21: TICK [1]
FUN 21: TICK [2]
FUN 21: TICK [3]
FUN 21: TICK [4]
FUN 21: TICK [5]
FUN 21: TICK [6]
FUN 21: TICK [7]
FUN 21: TICK [8]
FUN 21: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 22: INVOKE!
FUN 22: IN BURST[0]
FUN 22: TICK [1]
FUN 22: TICK [2]
FUN 22: TICK [3]
FUN 22: TICK [4]
FUN 22: TICK [5]
FUN 22: TICK [6]
FUN 22: TICK [7]
FUN 22: TICK [8]
FUN 22: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 23: INVOKE!
FUN 23: IN BURST[0]
FUN 23: TICK [1]
FUN 23: TICK [2]
FUN 23: TICK [3]
FUN 23: TICK [4]
FUN 23: TICK [5]
FUN 23: TICK [6]
FUN 23: TICK [7]
FUN 23: TICK [8]
FUN 23: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 24: INVOKE!
FUN 24: IN BURST[0]
FUN 24: TICK [1]
FUN 24: TICK [2]
FUN 24: TICK [3]
FUN 24: TICK [4]
FUN 24: TICK [5]
FUN 24: TICK [6]
FUN 24: TICK [7]
FUN 24: TICK [8]
FUN 24: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 25: INVOKE!
FUN 25: IN BURST[0]
FUN 25: TICK [1]
FUN 25: TICK [2]
FUN 25: TICK [3]
FUN 25: TICK [4]
FUN 25: TICK [5]
FUN 25: TICK [6]
FUN 25: TICK [7]
FUN 25: TICK [8]
FUN 25: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 26: INVOKE!
FUN 26: IN BURST[0]
FUN 26: TICK [1]
FUN 26: TICK [2]
FUN 26: TICK [3]
FUN 26: TICK [4]
FUN 26: TICK [5]
FUN 26: TICK [6]
FUN 26: TICK [7]
FUN 26: TICK [8]
FUN 26: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 27: INVOKE!
FUN 27: IN BURST[0]
FUN 27: TICK [1]
FUN 27: TICK [2]
FUN 27: TICK [3]
FUN 27: TICK [4]
FUN 27: TICK [5]
FUN 27: TICK [6]
FUN 27: TICK [7]
FUN 27: TICK [8]
FUN 27: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 28: INVOKE!
FUN 28: IN BURST[0]
FUN 28: TICK [1]
FUN 28: TICK [2]
FUN 28: TICK [3]
FUN 28: TICK [4]
FUN 28: TICK [5]
FUN 28: TICK [6]
FUN 28: TICK [7]
FUN 28: TICK [8]
FUN 28: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 0
FUN 29: INVOKE!
FUN 29: IN BURST[0]
FUN 29: TICK [1]
FUN 29: TICK [2]
FUN 29: TICK [3]
FUN 29: TICK [4]
```

Figure 30: Testing with `_USE_PROCESSES_` and `_TERMINATING_FUNCTIONS_` macros enabled (process isolation with termination, without round-robin scheduling)

```
phn@phn1011@PhanGowhan x + +
Copied 2 page directory entries
Setting up process stack at virtual address: <5243984>
esp = <5243824>
done
esp = <2099260>
done
Adding thread to ready queue: 1
Adding thread to ready queue: 2
Adding thread to ready queue: 3
STARTING THREAD 1 ...
Thread: 0
FUN 1 INVOKED!
FUN 1 IN BURST[0]
FUN 1: TICK [0]
FUN 1: TICK [1]
FUN 1: TICK [2]
FUN 1: TICK [3]
FUN 1: TICK [4]
FUN 1: TICK [5]
FUN 1: TICK [6]
FUN 1: TICK [7]
FUN 1: TICK [8]
FUN 1: TICK [9]
Resuming thread: 0
Yielding to thread: 1
Thread: 1
FUN 2 INVOKED!
FUN 2 IN BURST[0]
FUN 2: TICK [0]
FUN 2: TICK [1]
FUN 2: TICK [2]
FUN 2: TICK [3]
FUN 2: TICK [4]
FUN 2: TICK [5]
FUN 2: TICK [6]
FUN 2: TICK [7]
FUN 2: TICK [8]
FUN 2: TICK [9]
Resuming thread: 1
Yielding to thread: 2
```

Figure 31: Testing with `_USE_PROCESSES_` without `_TERMINATING_FUNCTIONS_` macros enabled (process isolation without termination, without round-robin scheduling)